

PaFEx: Compiler-Based Floating-Point Exception Detection for GPU Kernels with PAPIspecific Software-Defined Events

Abstract—As high-performance computing becomes progressively heterogeneous, the dependence upon vendor specific tools for numerical correctness has become an impediment to portability. Although modern GPUs comply with the IEEE 754 standard, the lack of practical native hardware support to raise and handle exceptions (special values like $\pm\infty$ or NaN) is a well-known architectural limitation. To embed portable numerical correctness across heterogeneous systems, we propose an architecture agnostic prototype based on LLVM-compiler infrastructure. This framework detects floating-point exceptions in GPU kernels at the Intermediate Representation (IR) level, instrumenting both device code and host code, strictly complying with the 2019 IEEE 754 standard. The host side is integrated with a customized PAPIspecific Software-Defined Events (SDE) based library, exposing the exception events to standard HPC logging and monitoring workflows. Currently implemented for NVIDIA CUDA kernels, this IR-level design can be extended fundamentally to port into GPU backends. Because it classifies exceptions by operation semantics, the tool reports both where exceptional values originate and where they propagate. The tool detects exceptions with a geometric-mean kernel-time overhead of 1.71x. We evaluated it on benchmarks from the Rodinia, PolyBench, SHOC, Parboil, and ECP XSbench proxy, using GPU-FPX as the baseline for comparison.

Index Terms—IEEE 754 exception, LLVM-instrumentation, PAPI, SDEs, runtime profiling.

I. INTRODUCTION

The scientific world is increasingly relying on heterogeneity for computations using diverse GPUs accelerators for highly parallel and data-intensive tasks. For such ubiquitous usage and managing this hardware diversity, the development of unified software ecosystems for hpc systems is encouraged for numerical correctness, optimization and hardware portability. However, to maximize performance and throughput, vendors of modern GPUs lack hardware-level exception tracking mechanism that is available in traditional CPUs. When floating point exceptional values- NaNs, INFs, subnormals occur, they propagate silently in GPU kernels. This causes a discrepancy in final results in long running simulations. Strict adherence to the IEEE standard for exceptions is essential to solve this, as it provides standardized clauses required to trace these events in the source code.

Where hardware support does exist; such as in AMD (MI200/MI300) [1], [2] and Intel (Xe-HPC) [3] architectures—it remains impractical for high-performance workloads. These accelerators utilize status registers that trap and halt execution upon detecting an exception. Yet iteratively trapping these exceptions by masking the exception site and relaunching the kernel introduces enormous amount of overhead that severely degrades performance [4].

Contrarily, the NVIDIA architectures completely avoid these status registers at the hardware level for performance reasons. Hence, researchers have relied for several years on software instrumentation to catch exceptions [5], [6], [7]. The existing state of the art tools operate exclusively at the binary level, leaning entirely on NVIDIA’s proprietary machine code [6], [7], which is vendor-specific. As scientific applications are designed to be portable, these diagnostic tools should be portable as well. Despite earlier tools, like FPChecker pioneered compiler-assisted exception tracking, development in this direction was abandoned in favor of binary-level analysis [5]. We re-introduce this compiler-level approach to bring attention to a unified framework that enforces IEEE 754 compliance across heterogeneous deployments. Although our long-term goal is a fully cross-vendor ecosystem, our current prototype specifically targets CUDA applications to demonstrate the viability of architecture-agnostic instrumentation.

Our framework, PaFEx is based on a dual-pass LLVM transformation strategy: a device-side pass injecting exception checks into the device code of GPU kernels, and a host-side pass instrumenting the host code with Performance Application Programming Interface (PAPI) Software Defined Events (SDEs) counters. With these PAPI SDE counters, we expose the exception counts directly as standard performance counters. This lets us precisely measure the performance impact of our instrumentation without needing specialized profiling software.

This paper makes the following contributions:

- **A compiler-level, vendor-neutral exception detector.** We revive the compiler-assisted approach with PaFEx, an LLVM IR tool that detects

IEEE 754-2019 floating-point exceptions through bit-level checking of target-independent integer operations rather than a vendor instruction set (§III).

- **Operand-side detection.** PaFEx tracks each exception from the operation’s operands, and attributes it to the originating site, distinguishing cases that result-only inspection misidentifies; $0/0$ (invalid) from $x/0$ (division by zero); and locating each to a `file:function:line`.
- **Profiling through PAPI SDE.** We publish exception counts as PAPIspecific Software-Defined Events using fixed-size arrays. This ensures the GPU-to-CPU data transfer cost is minimal and strictly constant, regardless of how many exceptions occur. As a result, the dominant overhead comes strictly from the exception checking on the device, not the reporting.

II. BACKGROUND

This section provides detail background of the underlying architecture of PaFEx. First, we outline the IEEE 754 standard (§II-A), based on which the exception detection design has been decided, along with NVIDIA’s compliance with the standard (§II-A1). Second, we discuss the LLVM compiler infrastructure (§II-B), and how its IR enables target-independent instrumentation. Finally, we introduce PAPIspecific Software-Defined Events (§II-C), a modern extension of the traditional performance-counter interface in the PAPI profiling architecture. This serves as our runtime library and standardized reporting mechanism for exposing exception counts.

A. IEEE 754 Exceptions and Default Handling

The IEEE 754-2019 standard [8] specifies that binary floating-point values are represented in three components: a sign bit, a biased exponent field, and a significand field. To manage conditions where arithmetic operations produce results that cannot be represented as normal numerical values, the standard categorizes these scenarios as exceptions, dividing them into five distinct classes: invalid operation, division by zero, overflow, underflow, and inexact. If any of these exceptions are signaled, IEEE 754 outlines two distinct paradigms for handling them: default (masked) handling and trapped handling.

By default, these exceptions are masked. When a masked exception is signaled, the system does not interrupt the program’s control flow. Instead, it raises the corresponding status flag to record the event and delivers a mathematically safe default result; such as a qNaN for an invalid operation or a properly signed ∞ for division by zero; allowing execution to proceed. Conversely, unmasking an exception enables a hardware trap that

halts execution to invoke a handler. Because taking a trap fundamentally disrupts the SIMT (Single Instruction, Multiple Threads) execution model and instruction pipelines of modern GPU architectures, GPU kernels overwhelmingly enforce masked exception handling. It is this strict adherence to the default masked behavior that facilitates the silent, unchecked propagation of numerical faults throughout long-running scientific simulations.

The standard distinguishes between two kinds of NaNs: Quiet NaNs (qNaNs) and Signaling NaNs (sNaNs). While qNaNs allow execution to continue by propagating through subsequent computations, sNaNs are designed to detect uninitialized variables or invalid memory accesses. An invalid operation exception is signaled when an operation has no mathematically defined floating-point result (e.g., ∞/∞ , $\infty - \infty$, $0 \times \infty$, $\sqrt{-1}$). Under default exception handling, this condition raises the invalid flag and delivers a qNaN as the result. Furthermore, the presence of NaN operands dictates specific exception behavior: operating on an sNaN almost always signals an invalid exception, whereas operating on a qNaN usually does not, except for operations that do not produce a floating-point result, such as conversions to integer formats or signaling comparisons.

Although similar in appearance due to a particular invalid operation (∞/∞), the division-by-zero exception is distinct. It occurs when a finite, nonzero operand is divided by zero. The default result is a signed infinity, where the sign is the XOR of the operand signs. Both invalid operation and division by zero are independent of the rounding mode. In contrast, rounding modes play a significant role in the remaining three exceptions.

If the operands in a floating-point operation are valid finite numbers, the system first computes the infinitely precise real result,

$$c = a + b.$$

This value is then subjected to a rounding operator,

$$\hat{c} = \circ_p^e(c),$$

where p denotes the working precision and e represents an unbounded exponent range. The rounded value $\hat{c}$ is the representable result in the target format.

If the magnitude of the exact result exceeds the largest representable finite value (i.e., $e > E_{\max}$), an overflow exception occurs. The default handling depends on the rounding mode: round-to-nearest typically produces $\pm\infty$, whereas round-toward-zero yields the maximum representable finite number. Overflow also implies that the result is inexact.

If the exponent of the rounded result is smaller than the minimum exponent of the format (i.e., $e < E_{\min}$), and the magnitude of the result is smaller than the smallest normalized number, the result lies in the subnormal

range. However, underflow is not triggered solely by tininess. IEEE 754 requires two conditions for underflow:

- 1) The result is tiny (its magnitude falls below the minimum normal threshold).
- 2) The rounded result is inexact, i.e., $\hat{c} \neq c$.

If the conversion to a subnormal value preserves the exact value (that is, $\hat{c} = c$), no underflow flag is raised despite the tiny magnitude. If precision is lost during rounding, both the Underflow and Inexact flags are set, and the system returns the subnormal result (or zero, if rounding yields zero). Finally, if no higher-priority exception occurs, the Inexact exception is signaled whenever the rounded result $\hat{c}$ differs from the infinitely precise result c . These rules define how a compliant system *should* behave; how faithfully a given GPU follows them, and where it is permitted to deviate, is what we consider next.

1) NVIDIA GPU Floating-Point Compliance:

NVIDIA GPUs of compute capability 2.0 and above implement IEEE 754 arithmetic for both single and double precision, including correctly rounded division and square root and hardware fused multiply-add [9]. The results of individual operations therefore match the standard. What the hardware does not provide is the standard’s signaling behavior: trap handlers for floating-point exceptions are not supported, and there is no status flag a program can read to learn that an invalid operation, division by zero, or overflow has occurred. An exceptional value is produced according to IEEE 754 and then propagates silently through the computation, which is the gap PaFEx is built to close.

B. LLVM Compiler Pass

LLVM compiles a program through a target-independent intermediate representation (IR): a static single-assignment (SSA) form in which each value is defined once. Floating-point arithmetic appears in the IR as explicit instructions; `fadd`, `fsub`, `fmul`, `fdiv`, `frem`, and the comparison `fcmp`; and as intrinsics for operations without a dedicated opcode, such as `llvm.sqrt` and the fused multiply-add `llvm.fma`. Because the IR is shared across back ends, a floating-point operation is expressed identically for any target device (NVIDIA, AMD, or CPU). PaFEx detects exceptions at this level, so its detection logic does not depend on any single instruction set.

For CUDA, Clang lowers each source file into two IR modules: a device module carrying the kernels, which the NVPTX back end compiles to PTX, and a host module containing the CPU code and the runtime glue that registers and launches those kernels. PaFEx instruments both, one pass per module. The passes load through LLVM’s pass-manager plugin interface and are invoked by name (`fp-exception` on the device

side, `fp-host-instrument` on the host side), so no change to the compiler itself is required. Operating at this level also preserves source information: Clang attaches `DILocation` debug metadata to each instruction, recording the originating file, function, and line. PaFEx reads this metadata to map each exception back to its source location. Once the code is compiled to assembly, that information is gone.

C. PAPI and specific Software-Defined Events

The Performance Application Programming Interface (PAPI) has served for nearly two decades as the standard abstraction layer for reading hardware performance counters in HPC [10]. Specific Software-Defined Events (SDE) extend PAPI so that a software layer can register its own internal quantities as counters, which are then read through the same interface as hardware events. A library calls `papi_sde_init` once and registers each counter with `papi_sde_register_counter`; an external tool reads it with the ordinary `PAPI_start`, `PAPI_read`, and `PAPI_stop` calls it already uses for hardware counters. Registering an existing variable this way adds no work to the library’s execution path; PAPI reads the variable only when the application asks for it; so exposing a counter that the code already maintains costs nothing at runtime.

Each registered counter declares a mode. A *delta* counter reports the change in its value over a measurement window, which suits monotonically increasing tallies; an *instantaneous* counter reports its current value, which suits a quantity such as a ratio that is meaningful only as read. PaFEx registers its exception tallies as delta counters Table III. Reporting exceptions through SDEs lets PaFEx fit existing practice. The counts appear under a named event namespace that any PAPI-based tool can read without modification, so exception data can be collected alongside hardware counters in the same monitoring run.

III. DESIGN AND IMPLEMENTATION

Our design is based on three goals, structured with three components: two LLVM transformation passes and a PAPI-based runtime library. The DevicePass instruments the device module with bit-level checks based on the IEEE 754 clauses for each floating-point operation and records exception counts in fixed-size device arrays. The HostPass transforms the host module to read those counters back, it registers them with the CUDA runtime and inserts the PAPI-SDE hooks that start, read, and stop the exception events. The runtime library reads the counters once the measured region ends and reports them as PAPI-specific Software-Defined Events (SDE). Figure 1 shows the pipeline; both passes operate on

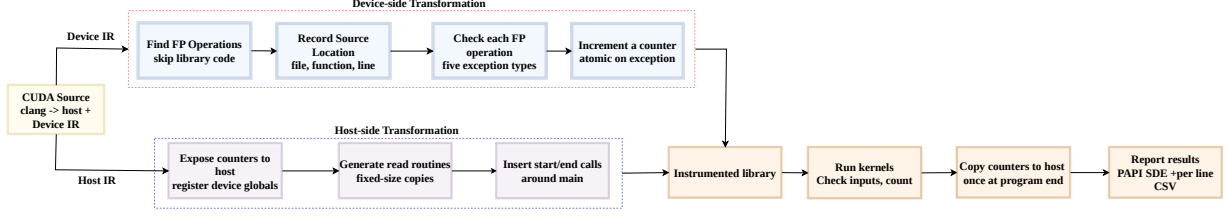


Fig. 1. Overview of the workflow of PaFEx

the LLVM IR emitted by Clang, and the instrumented modules link against the runtime and PAPI.

The three goals of the designs: (a) *Vendor-neutral detection*: classification uses only target-independent integer operations on the IEEE 754 bit pattern, so the same checks lower to any LLVM GPU backend and survive the fast-math transformations that would eliminate floating-point comparisons. (b) *Operand-side observation*: each exception is decided from the operation’s inputs, so its origin is recoverable; $0/0$ is reported as invalid while $x/0$ is division by zero. (c) *Bounded, standard reporting*: counts accumulate in fixed-size device arrays drained once per region and published through PAPI, so host cost is independent of how many exceptions occur and the results enter the existing performance-monitoring stack.

A. Device-Level IR Transformation

The detection mechanism is implemented as an LLVM module pass that specifically targets the `nvptx64` architecture, ensuring it only instruments compiled device code within a CUDA application. Because modern GPUs execute thousands of threads concurrently, naive instrumentation—such as injecting global atomic counters on every instruction—results in catastrophic memory serialization. To achieve high fidelity with low overhead, our pass uses a decoupled, two-stage pipeline centered on sparse Control Flow Graph (CFG) mutation, as outlined in Algorithm 1.

1) *Safe Collection Phase*: Modifying the intermediate representation (IR) while iterating through it can cause undefined behavior and compiler faults. Algorithm 1 avoids this by separating collection from mutation. The first stage finds all floating-point operations that could trigger an IEEE 754 exception. To ensure we only measure the user application’s actual numerical behavior, the pass explicitly filters out internal compiler and library calls, such as Thrust, CUB, `cuda::std`, and `libdevice` helpers. However, user-level functions that are inlined by the compiler remain fully instrumented. The pass extracts the source location, operation type, and precision for each valid instruction, pushing them to a worklist.

Algorithm 1 PaFEx Device Instrumentation

Require: $\mathcal{M}$: LLVM Device Module

Ensure: $\mathcal{M}$ instrumented with sparse exception counters

```

1:  $W \leftarrow \emptyset$  ▷ Worklist for safe mutation
2: ▷ Phase 1: Safe AST Traversal
3: for each Function  $F \in \mathcal{M}$  do
4:   if isDeviceLibrary( $F$ ) then ▷ Skip Thrust, CUB, and libdevice internals
5:     end if
6:   for each Instruction  $I \in F$  do
7:     if isFloatingPointOp( $I$ ) then
8:        $W \leftarrow W \cup \{I\}$ 
9:     end if
10:  end for
11: end for
12: ▷ Phase 2: Bitwise Evaluation & CFG Mutation
13: for each  $I \in W$  do
14:    $C_{\text{ex}} \leftarrow \text{GENERATEBITWISEPREDICATES}(I)$ 
15:   if  $C_{\text{ex}} \neq \text{null}$  then
16:     ▷ Create low-overhead fast path via SIMT divergence
17:      $BB_{\text{handler}} \leftarrow \text{SPLITBLOCKANDINSERTIFTHEN}(I, C_{\text{ex}})$ 
18:      $\text{INJECTATOMICRMW}(BB_{\text{handler}}, \text{fp\_counters})$ 
19:   end if
20: end for

```

2) *Architecture-Agnostic Bitwise Detection*: In the instrumentation phase, the pass settles the detection conditions strictly based on the IEEE 754 standard. To avoid the massive overhead of hardware-level context switches, we evaluate all mathematical conditions using purely integer bitwise operations. By bitcasting each floating-point operand into an integer of the same width and applying the precise hexadecimal masks shown in Table I, the pass builds lightweight boolean conditions inline. These predicates can quickly classify fundamental states—such as whether a value is zero, negative, sub-normal, infinite, a NaN, or at maximum finite magnitude.

TABLE I
BIT-FIELD MASKS USED FOR IEEE 754 CLASSIFICATION

Field	Binary32	Binary64
Sign	0x80000000	0x8000...0000
Exponent	0x7F800000	0x7FF0...0000
Mantissa	0x007FFFFF	0x000F...FFFF
Magnitude	0x7FFFFFFF	0x7FFF...FFFF
Quiet bit	0x00400000	0x0008...0000
Max. finite	0x7F7FFFFF	0x7FEF...FFFF

TABLE II
CATEGORIZATION OF EXCEPTIONS OF DIFFERENT OPERATIONS

Operation	Inv	Div0	Ovf	Unf	Sub
FAdd/FSub	✓		✓	✓	✓
FMul	✓		✓	✓	✓
FDiv	✓	✓	✓	✓	✓
FMA	✓		✓	✓	✓
FRem	✓				
FSqrt	✓				

Using these boolean predicates, the pass applies two different injection strategies depending on the mathematical context, as mapped in Table II:

- **Input-Specific Conditions:** Exceptions such as Division by Zero or Invalid Operations depend only on the input operands. Because these conditions are predetermined by the inputs, the pass evaluates and injects them right before the operation executes, preventing compiler optimization passes from improperly constant-folding the faults out of existence.
- **Result-Inspecting Conditions:** Exceptions like Overflow, Underflow, and Subnormal rely on the evaluated destination value. These are evaluated and injected immediately following the execution of the target instruction.

3) *Sparse CFG Injection:* Whenever a detector triggers a valid exception condition, the framework relies on CFG mutation rather than inline atomic execution. As shown in Phase 2 of Algorithm 1, the pass splits the basic block at the correct insertion point and adds an `AtomicRMWInst::Add` operation inside a new conditional branch. This operation writes directly to pre-allocated arrays in global device memory, simultaneously updating the aggregate total for that exception type and the specific, per-site counter linked to the original line of source code.

This design directly exploits the Single Instruction, Multiple Thread (SIMT) architecture of the GPU. If the exception predicate evaluates to false for every thread within a warp, the warp entirely bypasses the handler block. This sparse branching mechanism ensures that global memory contention only occurs when a mathematical anomaly is definitively proven, isolating the overhead strictly to failing code paths.

B. Host-Side Transformation

PaFEx utilizes a host-side LLVM pass to extract exception metrics from the GPU without requiring developers to modify their source code. As outlined in Algorithm 2, this pass automatically handles two critical tasks: linking memory spaces and injecting the PAPI profiling lifecycle hooks.

First, the pass establishes a transparent memory bridge between the device and the host. It scans the host code for the CUDA initialization sequence (`__cuda_register_globals`) and creates host-side “shadow” variables for the device’s tracking arrays. It binds these variables using the `__cudaRegisterVar` API. The pass then automatically generates extern “C” accessor functions that perform `cudaMemcpyFromSymbol` operations. This creates a standard C-interface, allowing external monitoring tools like PAPI to read the GPU data natively.

Second, the pass guarantees that the profiling runtime starts and stops correctly. It locates the application’s main function and inserts an initialization hook (`fp_instrument_init`) at the very beginning. It also injects a finalization hook (`fp_instrument_finalize`) immediately before every return statement. This ensures that all exception counters are safely flushed to disk before the program terminates.

C. PAPI SDE Library and Runtime Driver

Standard hardware counters cannot track software-emulated IEEE 754 exceptions. To expose these metrics to standard profiling workflows, PaFEx implements a custom Performance APISpecific Software-Defined Events (PAPI SDE) runtime library. During initialization, the runtime registers a dedicated namespace (`CUDA_FP_ANALYSIS`) containing the counters listed in Table III. All events are configured as `DELTA` accumulators, actively tracking both foundational exception occurrences (e.g., division by zero, underflow) and derived counters (e.g., total operations, kernels launched).

TABLE III
SDE COUNTERS

Counter	Type
FP_DIVBYZERO	DELTA
FP_INVALID	DELTA
FP_OVERFLOW	DELTA
FP_UNDERFLOW	DELTA
DENORMAL_PRODUCED	DELTA
KERNELS_LAUNCHED	DELTA
OPERATIONS_TOTAL	DELTA
TOTAL_EXCEPTIONS	DELTA

A dedicated driver synchronizes the GPU memory with the PAPI framework. When the `fp_instrument_read_and_publish` hook

Algorithm 2 Host Pass: Memory Linkage and PAPI Injection

Require: $\mathcal{M}_{host}$: Host-side module being compiled

Ensure: $\mathcal{M}_{host}$ augmented with GPU memory bindings and telemetry hooks

```
1: if module is a device target then return
2:        $\triangleright$  Phase 1: Establish Memory Linkage
3: if  $\mathcal{M}_{host}$  contains function __cuda_register_globals then
4:   for each tracking array (e.g., fp_counters) do
5:     Create a host-side shadow variable
6:     Inject __cudaRegisterVar to bind shadow to device memory
7:     Emit an extern "C" accessor using cudaMemcpyFromSymbol
8:   end for
9: end if
10:       $\triangleright$  Phase 2: Inject Lifecycle Hooks
11: if  $\mathcal{M}_{host}$  contains function main then
12:   Inject call to fp_instrument_init at the beginning of main
13:   for each return instruction in main do
14:     Inject call to fp_instrument_finalize immediately before returning
15:   end for
16: end if
```

is called, the driver immediately executes a `cudaDeviceSynchronize()` command. This acts as a safety barrier. It guarantees that all running GPU kernels have finished tallying their exceptions in global memory before the CPU attempts to read them, preventing data races. Once the GPU is idle, the driver calls the generated extern "C" functions to copy the data from the device back to the host, and pushes those final numbers into the PAPI SDE registry.

At application termination, the `fp_instrument_finalize` hook safely stops PAPI and triggers a final data dump. This provides a dual-output reporting strategy: the PAPI interface outputs high-level aggregate statistics for general workload monitoring, while the driver simultaneously exports the dense `fp_site_counters` array to a CSV file for granular, line-by-line diagnostic tracing.

D. Compilation and Integration Strategy

To instrument CUDA applications without modifying user source code, PaFEx intercepts the standard LLVM compilation pipeline. Because the proprietary `nvcc` compiler abstracts away the separation of host and

device code, PaFEx uses Clang to manually replicate and decouple this trajectory. The framework supports both single-translation-unit (Single-TU) and complex multi-translation-unit (Multi-TU) builds.

1. Single-TU Instrumentation (Core Pipeline)

The foundational strategy operates on a single CUDA source file by explicitly splitting the compilation into two distinct paths, followed by a unified linking phase:

- **Device Path:** The device code is compiled to unoptimized LLVM IR (`-O0`) and linked with `libdevice`. A critical pre-optimization step (always-inline, mem2reg, instcombine) is then applied to expose scalar floating-point operations hidden behind loads and abstract function calls. After applying the `DevicePass` instrumentation, the IR is lowered to a CUDA fatbinary. Aggressive optimizations (`-O2`) are strictly deferred to this final code generation phase to prevent the compiler from reordering or eliminating the injected atomic tracking operations.
- **Host Path:** The host code is similarly compiled to LLVM IR. To successfully link the two memory domains—mimicking the internal behavior of `nvcc`—the newly generated device fatbinary is explicitly embedded into the host code using Clang’s `-fcuda-include-gpubinary` flag. The `HostPass` is then applied to inject the memory linkage and lifecycle hooks before compiling to the final host object file.
- **Runtime Linking:** The instrumented host object file is linked against the pre-compiled PAPI SDE runtime objects (`fp_sde_counters.o`, `fp_sde_driver.o`) and the external PAPI/SDE libraries. This final step binds the injected lifecycle hooks to the active monitoring framework.

2. Multi-TU Integration

For complex applications spanning multiple files, PaFEx adapts the core pipeline to ensure cross-module kernel resolution. The device code for each source file is individually compiled, linked with `libdevice`, and pre-optimized to collapse heavy C++ template abstractions (e.g., Thrust, CUB) into concrete device functions. These unoptimized bitcodes are merged (`llvm-link`) before the `DevicePass` is applied globally. To guarantee that auto-generated host stubs can properly resolve device kernels across TU boundaries, the unified device fatbinary is explicitly embedded into *every* host translation unit before applying the `HostPass`. The resulting host objects are then linked with the PAPI SDE runtime, identical to the Single-TU strategy.

IV. RESULTS EVALUATION

A. Experimental Setup

We conduct all experiments on a node equipped with an NVIDIA A100-SXM4-40GB GPU (Ampere, sm_80) and a dual-socket AMD EPYC 7662 processor. PaFEx is built using LLVM/Clang 22.1.5 and CUDA 12.8, with the PAPI 7.3.0 specific Software-Defined Events (SDE) component.

Device code is instrumented at `-O0 -g` following a fixed canonicalization pass (`always-inline`, `mem2reg`, `instcombine`) that simplifies IR structure without altering the underlying floating-point operations. We use this same instrumentation configuration for both detection accuracy validation and overhead measurements. We evaluate PaFEx using a suite of CUDA programs drawn from Rodinia, PolyBench, Parboil, SHOC, and XSBench, filtering for kernels that perform floating-point arithmetic. We compare against GPU-FPX [7], run with NVBit 1.5.5.

B. Detection Validation

We validate PaFEx in two stages: micro-benchmarks targeting specific conditions in the IEEE 754 standard, and HPC benchmarks drawn from production suites.

a) Microbenchmarks.: PaFEx detects the expected exception class in 14 of 16 microbenchmark tests (Table IV). Because checks operate on operands rather than result registers, a kernel performing both $0/0$ and $x/0$ is reported as one invalid and eight divide-by-zero events; a distinction that result-register classification conflates. The two non-detections are documented limitations rather than misclassifications: `logb` lowers to a device-library call outside the instrumented IR-operation set, and `uf_denormal` operates on pre-existing subnormal inputs, so no normal-to-subnormal underflow event occurs at an instrumented site.

b) HPC benchmarks.: We then evaluate PaFEx across programs from Rodinia, PolyBench, Parboil, SHOC, and XSBench (Table V). Counts are distinct static source locations classified at the originating operation. Five benchmarks are well-conditioned and correctly report no exceptions, serving as negative controls; notably XSBench executes on the order of 10^9 instrumented operations with confirmed counter readback yet raises nothing, demonstrating the absence of false positives at scale.

The remaining benchmarks expose distinct exception profiles. LU and GRAMSCHM each surface a single invalid-operation site. `myocyte` (FP64) yields the richest profile; 14 invalid, 18 divide-by-zero, and 9 underflow sites; arising from its stiff rate equations. S3D produces overflow together with many underflow and subnormal sites in its wide-dynamic-range reaction kernels. The Parboil stencil produces no exceptions on a uniform field

Microtest	IEEE clause	Expected	Detected (counts)
<i>Invalid operation</i>			
<code>fma_zero_inf</code>	7.2	invalid	✓ invalid=5
<code>inf_sub</code>	7.2	invalid	✓ invalid=4
<code>mul_zero_inf</code>	7.2	invalid	✓ invalid=4
<code>sqrt_invalid</code>	7.2	invalid	✓ invalid=5
<code>inv_div</code>	7.2	invalid	✓ invalid=5, divzero=1 [†]
<i>Divide by zero</i>			
<code>divByZero</code>	7.3	divzero	✓ divzero=8, invalid=1 [†]
<code>logB_test</code>	7.3	divzero	✓ divzero=1
<i>Overflow</i>			
<code>round_nearest</code>	7.4	overflow	✓ overflow=4
<code>round_negative</code>	7.4	overflow	✓ overflow=4
<code>round_positive</code>	7.4	overflow	✓ overflow=4
<code>round_zero</code>	7.4	overflow	✓ overflow=4
<i>Underflow / subnormal</i>			
<code>uf_basic</code>	7.5	underflow	✓ underflow=5, sub=5
<code>uf_gradual</code>	7.5	underflow	✓ underflow=7, sub=6
<code>uf_iterative</code>	7.5	underflow	✓ underflow=1, sub=1
<code>uf_rounding</code>	7.5	underflow	✓ underflow=13, sub=11

[†]Mixed-class test; every operation classified correctly. [‡]Operates on pre-existing subnormals.

TABLE IV
VALIDATION ON MICROBENCHMARKS WITH KNOWN IEEE-754 BEHAVIOR.

TABLE V
EXCEPTION DETECTION ON HPC BENCHMARKS

Benchmark	Prec.	Inv.	DivZero	Ovf.	Unf.	Subn.
LU	FP32	1	0	0	0	0
GRAMSCHM	FP32	1	0	0	0	0
<code>myocyte</code>	FP64	14	18	0	9	0
S3D	FP32	0	0	7	127	13
stencil	FP32	0	0	0	4	4
<code>cfid</code>	FP32	0	0	0	11	11
<code>sgemm</code>	FP32	0	0	0	0	0
<code>gaussian</code>	FP32	0	0	0	0	0
<code>hotspot</code>	FP32	0	0	0	0	0
FFT	FP32	0	0	0	0	0
XSBench	FP64	0	0	0	0	0

but generates subnormals at four sites under a synthetic field near the FP32 minimum normal (the canonical Parboil dataset being unavailable).

For the two benchmarks where both tools detect exceptions (`myocyte`, S3D), PaFEx and GPU-FPX agree on the exception classes raised, corroborating detection. The two tools surface different per-site counts because they classify at different points in the operation: PaFEx inspects operands at the IR-level instrumented operation, while GPU-FPX inspects the result register at the SASS level. This is a methodological difference, not a discrepancy in correctness.

C. Instrumentation Overhead

We measure overhead as a slowdown ratio: instrumented GPU kernel time divided by baseline kernel time, isolating instrumentation cost from host-side setup

TABLE VI
RUNTIME OVERHEAD: INSTRUMENTED/BASELINE GPU KERNEL
TIME (NSIGHT SYSTEMS), DETECTION-ONLY CONFIGURATION.

Benchmark	Baseline	Slowdown
LU	71.5 ms	1.35×
gaussian	0.19 ms	1.52×
hotspot	0.17 ms	2.03×
GRAMSCHM	25.0 s	2.08×
Geomean		1.71×

and data transfer. In this configuration PaFEx checks every floating-point operation but updates a counter only when an exceptional condition holds. Across the four benchmarks in Table VI, PaFEx incurs a geometric-mean slowdown of $1.71\times$, ranging from $1.35\times$ on LU to $2.08\times$ on GRAMSCHM. Because a check is inserted at every instrumented operation, the dominant cost is per-operation, so overhead scales with floating-point density: kernels that execute more floating-point operations require proportionally more checks. XSBench illustrates this worst case at $16.7\times$; it raises no exceptions (Table V), so its counters never increment and the entire cost is the per-operation bit-pattern checks applied across its $\sim 10^9$ floating-point operations.

PaFEx reports its results to the host through PAPI SDE counters. Instead of sending data to the host every time an exception occurs, it keeps all counts in device memory while the program runs and copies them back just once, at the end, in a single small transfer. We measured this readback at about $77\mu\text{s}$, and a one-time startup cost of under a second when the instrumentation is first initialized. Both are fixed: they do not grow with the size of the workload or the number of kernels, and they fall outside the kernel-time measurements above. A binary-instrumentation tool, by contrast, sends a record to the host for each exceptional value, so its host cost grows with the number of exceptions.

V. RELATED WORK

The mechanism to detect exceptions in floating point arithmetic is available in traditional CPU, unlike in modern GPUs. Thus, research on building software tools for detecting these events in GPU has been adopted throughout the years.

One of the first influential works in floating-point optimization is GPUMixer [11], which utilizes static analysis and dynamic shadow computations to apply safe mixed-precision substitutions. GPUMixer emphasizes reducing execution time and improving throughput by minimizing type conversions while preserving accuracy. However, it does not explicitly focus on detecting exceptions or identifying the root causes of numerical faults. Following this optimization-centric focus, CUD-

AAdvisor [12] explored a profiling-oriented approach to analyze bottlenecks through memory and control-flow divergence analysis. Although not designed for floating-point behavior, CUDAAAdvisor [12] demonstrated the feasibility of using LLVM-based tooling to extract fine-grained runtime insights from CUDA kernels, influencing our decision to adopt LLVM IR for instrumentation. The need for exception-specific profiling was pioneered by FPChecker [5], which introduced LLVM-based instrumentation to detect anomalous floating-point values such as NaNs and infinities. Early versions of FPChecker [5] provided comprehensive instrumentation for both CPU and CUDA applications. However, recent development of FPChecker has refocused exclusively on CPU-based analysis. To address the challenges of GPU instrumentation, the original authors of FPChecker [5] have transitioned to binary-level analysis tools such as BinFPE [6] and GPU-FPX [7]. BinFPE [6] leverages NVIDIA’s NVBit dynamic binary instrumentation framework to identify floating-point anomalies directly at the SASS (assembly) level, allowing the analysis of binaries without recompilation. GPU-FPX extends this by focusing on detecting anomalous values and correlating them with program execution contexts using coalesced stack trace graphs. These tools operate on raw assembly to ensure execution-level accuracy and avoid compiler optimization artifacts.

While binary instrumentation offers high accuracy, it introduces significant challenges in debugging traceability (mapping faults back to source code lines is difficult in stripped binaries) and portability (tight coupling to specific GPU ISAs like NVPTX/SASS). This cross-vendor fragmentation is further highlighted by recent work like FloatGuard [4], which extends exception detection to AMD GPUs but remains heavily reliant on vendor-specific hardware mechanisms. Our work revisits the compiler-assisted approach, addressing its historical limitations—such as false positives in optimized code—through improved control-flow handling (Algorithm Captions) and integrating it with the PAPI SDE [10] ecosystem. This enables modern, performance-correlated reliability analysis that retains the source-level visibility required for effective debugging, bridging the gap between binary-level accuracy and source-level usability.

While tools like GPUCC [13] and FlowFPX [14] have demonstrated LLVM’s broader utility in GPU workflows or provided high-level debugging insights, they do not target low-level floating-point exceptions directly. In contrast, our approach treats IEEE 754 exceptions as first-class instrumentation targets within the LLVM IR. By encoding clause-specific IEEE 754 [15] logic and rounding-mode effects directly into the intermediate representation, we enable scalable, IEEE standard-aware

exception tracking that is both accurate and developer-friendly.

VI. CONCLUSION AND FUTURE WORK

We presented PaFEx, a compiler-level floating-point exception detector for CUDA that analyzes LLVM IR rather than hardware-specific machine code. This approach lets us see exception behavior at the source-operation level, but it has two trade-offs. First, PaFEx cannot inspect pre-compiled vendor libraries such as cuBLAS, since their machine code is not available to our pass. Second, because PaFEx records operations before the final compiler optimizations run, the IR-level counts may differ slightly from the instructions that actually execute on the hardware. Despite these limitations, PaFEx’s target-independent detection logic provides a strong foundation for future work.

We plan to extend PaFEx along four directions. (1) We will use the compile-time site information PaFEx already records to produce granular line-by-line reports, so users can see which source operations contributed to each exception. (2) We will study how standard compiler optimizations affect both PaFEx’s counts and the underlying exception behavior, comparing instrumentation at $-O0$, $-O1$, $-O2$, and $-O3$. (3) We will investigate whether our compiler-based approach can be extended to handle heavily templated libraries like Thrust and CUB, which today are analyzed only at the binary level, (4) We will port PaFEx to AMD GPUs through the HIP/ROCm backend.

Our long-term goal is a unified, portable exception detector for GPU programs that follows a standard compilation pipeline and verifies numerical behavior against the IEEE 754 standard. Achieving cross-vendor portability would let us empirically expose how differing internal hardware designs cause architectures that claim IEEE compliance to nonetheless produce divergent numerical results under exceptional conditions.

REFERENCES

- [1] Advanced Micro Devices, Inc., *AMD Instinct MI200 Instruction Set Architecture: Reference Guide*, 2022, revision 1.01. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/instruction-set-architectures/instinct-mi200-cdna2-instruction-set-architecture.pdf>
- [2] —, *AMD Instinct MI300 Instruction Set Architecture: Reference Guide*, 2024. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/instruction-set-architectures/amd-instinct-mi300-cdna3-instruction-set-architecture.pdf>
- [3] Intel Corporation, *Intel Iris Xe Graphics Open Source Programmer’s Reference Manual (PRM) - Volume 9: Render Engine*, 2021, details the architectural register file (ARF), including Control Register 0 (cr0) and State Register 0 (sr0) for floating-point exception handling. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/graphics-for-linux/developer-reference/1-0/hardware-specs.html>
- [4] D. Miao, I. Laguna, and C. Rubio-González, “Floatguard: Efficient whole-program detection of floating-point exceptions in amd gpus,” in *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*, 2025, pp. 1–12.
- [5] I. Laguna, T. Tirpankar, X. Li, and G. Gopalakrishnan, “FPChecker: Floating-Point Exception Detection Tool and Benchmark for Parallel and Distributed HPC,” in *IEEE International Symposium on Workload Characterization, IISWC 2022, Austin, TX, USA, November 6-8, 2022*. IEEE, 2022, pp. 39–50. [Online]. Available: <https://doi.org/10.1109/IISWC55918.2022.00014>
- [6] I. Laguna, X. Li, and G. Gopalakrishnan, “BinFPE: accurate floating-point exception detection for GPU applications,” in *Proceedings of the 11th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*, 2022, pp. 1–8.
- [7] X. Li, I. Laguna, B. Fang, K. Swirydowicz, A. Li, and G. Gopalakrishnan, “Design and Evaluation of GPU-FPX: A Low-Overhead tool for Floating-Point Exception Detection in NVIDIA GPUs,” in *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing, HPDC 2023, Orlando, FL, USA, June 16-23, 2023*, A. R. Butt, N. Mi, and K. Chard, Eds. ACM, 2023, pp. 59–71. [Online]. Available: <https://doi.org/10.1145/3588195.3592991>
- [8] “Ieee standard for floating-point arithmetic,” *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, 2019.
- [9] N. Whitehead and A. Fit-Florea, “Floating point and IEEE-754 compliance for NVIDIA GPUs,” *Nvidia Whitepaper*, 2017.
- [10] A. Danalis, H. Jagode, T. Herault, P. Luszczek, and J. Dongarra, “Software-defined Events through PAPI,” in *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2019, pp. 363–372.
- [11] I. Laguna, P. C. Wood, R. Singh, and S. Bagchi, “Gpumixer: Performance-driven floating-point tuning for gpu scientific applications,” in *International Conference on High Performance Computing*. Springer, 2019, pp. 227–246.
- [12] D. Shen, S. L. Song, A. Li, and X. Liu, “Cudaadvisor: Llvm-based runtime profiling for modern gpus,” in *Proceedings of the 2018 International Symposium on Code Generation and Optimization*, 2018, pp. 214–227.
- [13] J. Wu, A. Belevich, E. Bendersky, M. Heffernan, C. Leary, J. Pienaar, B. Roune, R. Springer, X. Weng, and R. Hundt, “gpcc: an open-source gpgpu compiler,” in *Proceedings of the 2016 International Symposium on Code Generation and Optimization*, 2016, pp. 105–116.
- [14] T. Allred, X. Li, A. Wiersdorf, B. Greenman, and G. Gopalakrishnan, “Flowfpx: Nimble tools for debugging floating-point exceptions,” *arXiv preprint arXiv:2403.15632*, 2024.
- [15] J.-M. Muller, N. Brisebarre, F. De Dinechin, C.-P. Jeannerod, V. Lefevre, G. Melquiond, N. Revol, D. Stehlé, S. Torres *et al.*, *Handbook of floating-point arithmetic*. Springer, 2018, vol. 1.